

Kreacijski dizajn paterni

Singleton

Uloga Singleton paterna je da osigura da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci. U našem sistemu, klasa `DataContext` predstavlja kontekst baze podataka koji sadrži kolekcije svih entiteta sistema (`korisnici`, `pitanja`, `predmeti`, `sesije`, `feedbacki`, `leaderboard` i dr.). Bez Singleton paterna, svaki kontroler bi mogao kreirati vlastitu instancu `DataContext`-a, što bi vodilo do nekonzistentnosti podataka i nepotrebnog trošenja resursa.

To možemo implementirati:

- kreiranjem klase `DataContext` sa privatnim konstruktorom kako bi se spriječilo direktno instanciranje izvana;
- definiranjem privatnog statičkog atributa `instanca`: `DataContext` koji čuva jedinu instancu klase;
- dodavanjem javne statičke metode `getInstanca()`: `DataContext` koja pri prvom pozivu kreira instancu, a pri svakom narednom vraća već postojeću;
- osiguravanjem da svi kontroleri (`KorisnikController`, `PitanjeController`, `KvizController`, `FeedbackController`, `PredmetController`) pristupaju bazi podataka isključivo putem `DataContext.getInstanca()`.

Korištenje: Unutar sistema možemo koristiti Singleton kako bismo osigurali da postoji samo jedna otvorena konekcija prema bazi podataka tokom cijelog životnog vijeka aplikacije. Na taj način se osigurava konzistentnost stanja podataka kroz cijelu aplikaciju, a resursi se efikasnije koriste jer se izbjegava višestruko kreiranje skupih objekata za upravljanje bazom podataka.

Prototype

Prototype patern obezbjeđuje interfejs za konstrukciju objekata kloniranjem jedne od postojećih prototip instanci i modifikacijom te kopije. U našem sistemu postoje tri tipa korisnika: `Student`, `Moderator` i `Administrator`, od kojih svaki nasljeđuje apstraktnu klasu `Korisnik`. Svaki tip korisnika ima unaprijed definisan skup prava pristupa i podrazumijevanih postavki koje se ponavljaju za sve korisnike iste uloge.

To možemo implementirati:

- definiranjem metode `clone()` unutar apstraktne klase `Korisnik` koja vraća kopiju objekta;
- kreiranjem prototip instanci za svaki tip korisnika (`studentPrototip`, `moderatorPrototip`,

`administratorPrototip`) sa unaprijed postavljenim pravima i postavkama karakterističnim za svaku ulogu;

- implementacijom konkretnih metoda kloniranja u podklasama `Student`, `Moderator` i `Administrator`;
- korištenjem odgovarajućeg prototipa pri registraciji novog korisnika, uz izmjenu samo specifičnih podataka poput korisničkog imena i email adrese.

Korištenje: Prototype patern se može koristiti pri registraciji novih korisnika sistema. Kada administrator dodijeli ulogu novom korisniku, sistem klonira odgovarajući prototip (npr. `studentPrototip`) i izmijeni samo specifične podatke kao što su `Username` i `Email`. Na ovaj način pojednostavljuje se proces kreiranja korisnika, osigurava konzistentnost između korisnika iste uloge i izbjegava ponavljanje logike postavljanja podrazumijevanih vrijednosti.

Factory Method

Factory Method patern omogućava kreiranje objekata na način da podklase, na osnovu informacije od strane klijenta ili tekućeg stanja, odluče koju klasu instancirati. U našem sistemu, apstraktna klasa `Feedback` ima dvije konkretne podklase: `PrijedlogPitanja` i `PrijavaGreske`. Bez ovog patern, svaki dio sistema koji treba kreirati `Feedback` objekat morao bi direktno instancirati konkretnu podklasu, što bi dovelo do rasute logike kreiranja i povećane sprege između komponenti.

To možemo implementirati:

- definiranjem apstraktne klase `FeedbackFactory` sa apstraktnom metodom `kreiraiFeedback(tip: TipFeedback): Feedback`;
- implementacijom konkretne klase `PrijedlogPitanjaFactory` koja nadjačava metodu `kreiraiFeedback()` i vraća instancu klase `PrijedlogPitanja`;
- implementacijom konkretne klase `PrijavaGreskeFactory` koja nadjačava metodu `kreiraiFeedback()` i vraća instancu klase `PrijavaGreske`;
- osiguravanjem da `FeedbackController` komunicira isključivo sa `FeedbackFactory` i nije svjestan konkretnih podklasa.

Korištenje: Kada student inicira slanje povratne informacije, sistem na osnovu odabranog tipa (`PRIJEDLOG_PITANJA` ili `PRIJAVA_GRESKE`) poziva odgovarajuću fabriku. Klijentski kod ne mora voditi računa o tome iz koje konkretno klase će biti instanciran objekat, čime se uklanjaju grananja u raznim dijelovima koda, a logika odlučivanja smješta na jedno mjesto. Dodavanje novog tipa feedbacka u budućnosti zahtijeva samo kreiranje nove fabrike i nove podklase, bez izmjene postojećeg koda.

Abstract Factory

Abstract Factory patern omogućava kreiranje porodica povezanih objekata bez specificiranja konkretnih klasa. Koristi se kada postoji hijerarhija objekata koja enkapsulira različite platforme sastavljene od grupe povezanih objekata. Razmatrajući naš sistem za provjeru

znanja, pojava različitih varijacija kvizova za različite tipove provjere znanja (priprema i simulacija) mogla bi opravdati korištenje ovog paterna.

To možemo implementirati:

- definiranjem apstraktne fabrike `KvizFactory` koja sadrži metode za kreiranje objekata `KvizSesija` i `KvizSesijaPitanje`;
- implementacijom konkretne fabrike `PripremaFactory` koja kreira objekte prilagođene načinu rada bez vremenskog ograničenja i s trenutnim prikazom tačnih odgovora;
- implementacijom konkretne fabrike `SimulacijaFactory` koja kreira objekte prilagođene simulaciji s vremenskim ograničenjem i bez otkrivanja tačnih odgovora tokom trajanja kviza.

Korištenje: Pošto naš sistem podržava dva načina rada - pripremu i simulaciju kviza - Abstract Factory patern bi omogućio kreiranje odgovarajućih varijanti objekata za svaki način rada bez izmjene klijentskog koda. Međutim, uzimajući u obzir trenutnu složenost sistema i broj varijacija, ovaj patern donosi određenu kompleksnost čija je primjena tek djelimično opravdana u trenutnoj fazi razvoja.

Builder

Builder dizajn patern koristan je za kreiranje složenih objekata korak po korak. Primjenjuje se kada objekat ima mnogo opcionalnih parametara ili kada je potrebna fleksibilnost u kreiranju objekta. U našem sistemu, objekat `KvizSesija` ima više parametara koji mogu biti opcionalni ili zavisni od konteksta: broj pitanja, vremensko ograničenje, odabrana oblast i status sesije. Bez Builder paterna, konstruktor bi imao veliki broj parametara što otežava čitljivost i održavanje koda.

To možemo implementirati:

- kreiranjem klase `KvizSesijaBuilder` koja sadrži iste attribute kao `KvizSesija`;
- implementacijom fluent metoda za postavljanje svakog parametra: `setBrojPitanja()`, `setVremenskoOgranicenje()`, `setOblastID()`, `setStudentID()`;
- dodavanjem metode `build()`: `KvizSesija` koja na kraju kreira i vraća potpuno konfigurisanu instancu `KvizSesija`;
- korištenjem `KvizSesijaBuilder`-a unutar `KvizController`-a umjesto direktnog instanciranja `KvizSesija` objekta.

Korištenje: Builder patern bi se koristio pri pokretanju kviza, kada `KvizController` treba kreirati `KvizSesija` objekat. Za simulaciju kviza postavljaju se svi parametri uključujući vremensko ograničenje, dok se za pripremu gradiva vremensko ograničenje izostavlja. Na ovaj način se omogućava fleksibilno i pregledno kreiranje objekata s opcionalnim parametrima, bez pretrpavanja konstruktora i bez komplikovanja osnovne strukture objekta.